

Seen

Section - A

1. What do you mean by greedy?

Ans. It means making locally optimal choice at each step, with hope that it will lead to a globally optimal solution.

2. What is difference betn graph and tree?

Ans. main difference is there structure.

A graph is a collection of nodes connected by edges, where edges can go in any direction and can have no of connection.

Tree is a specific type of graph that has ~~horizon~~ hierarchical structure.

3. Write about the data structure of graph.

Ans. A graph is a data structure comprising vertices (nodes) and edges. vertices represent entities and edges denote relationship between entities. The structure can be represented using an adjacency matrix or an adjacency list. Graphs can be directed or undirected and may have weights on edges.

4. What do you mean DAG?

Ans. A DAG (Directed Acyclic Graph) is a graph data structure with directed edges, meaning each edge has a specific direction from one vertex to another. The critical property of a DAG is that it is acyclic, ensuring there are no cycles or loops in the graph.

5. What do you mean by dynamic programming?

Ans. It is a technique used to solve complex problem by breaking them down into smaller overlapping subproblems. It's like solving a big problem by solving series of smaller related problems and then combining their solutions.

• section - B

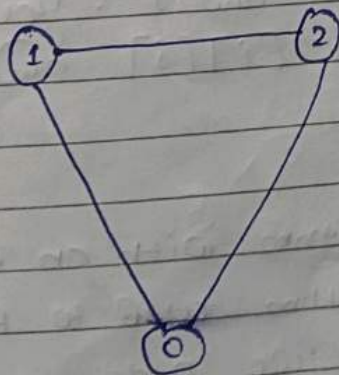
(1). Discuss How to represent graph storage using Adjacency matrix.

Ans. Adjacency matrix is 2D matrix that represents connection betⁿ nodes in graph.

• Representation of Undirected Graph to matrix.

- if there is an edge from vertex i to j . mark $adjMat[i][j]$ as 1.
- if there is no edge from vertex i to j . Mark $adjMat[i][j]$ as 0.

→ initially, the entire matrix is initialized to 0. if there is an edge from source to destination, we insert 1 to both cases ($adjMat[source]$ and $adjMat[destination]$) because we can go either way.

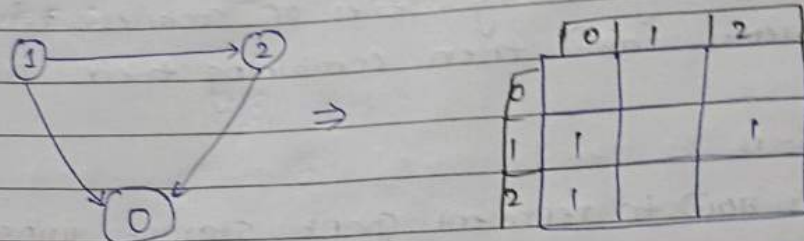


Undirected graph

	0	1	2
0		1	1
1	1		1
2	1	1	

Adjacency Matrix

- Representation of Directed Graph to Adjacency Matrix:-
- Initially, the entire matrix is initialized to 0. If there is an edge from source to destination, we insert 1 for that particular adjMat[destination].



Directed Graph

Adjacency Matrix

- Here How it works -
- Create a matrix -
- matrix size is determined by no. of nodes in graph.
- initialize the matrix.
- initially all the cells in matrix are set to 0 or false indicating no connection betⁿ nodes.
- Represent Connections -
- For each edge in graph, you update corresponding cells in matrix. If there is edge b/w node i and j , you set the value of matrix $[i][j]$ and $[j][i]$ to 1 or true.

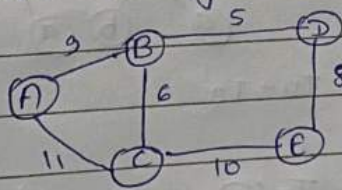
2. Explain Dijkstra's algorithm with an example.

Ans. it is a popular algorithm used to find Shortest Path between two nodes in graph.

Procedure :-

- Start by assigning a tentative distance between value to every node in graph. set initial node distance to 0 and all other to ∞ .
- Mark initial node as visited and set it as current node.
- for current node, consider all of its adjacent unvisited neighbours and calculate tentative distance through current node. Compare these calculated distances with current assigned value and update tentative distances if they are smaller.
- Once you have considered all unvisited neighbour of current node won't be checked again.
- If destination node has been marked visited or if smallest tentative distance among unvisited nodes is ∞ , then stop.
- Otherwise, select unvisited node with smallest distance as current node and go back to step 3.

example:



Starting from A, it would find shortest distⁿ to all nodes

Q 3. Construct DAG For

$$T_1 = a + b$$

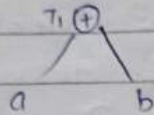
$$T_2 = a - b$$

$$T_3 = T_1 * T_2$$

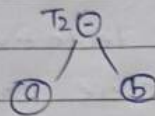
$$T_4 = T_1 - T_3$$

$$T_5 = T_4 + T_3$$

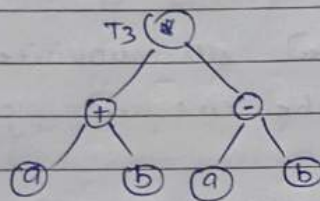
Ans Step 1: $T_1 = a+b$



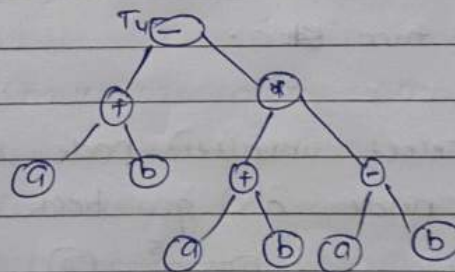
• Step 2: $T_2 = a-b$



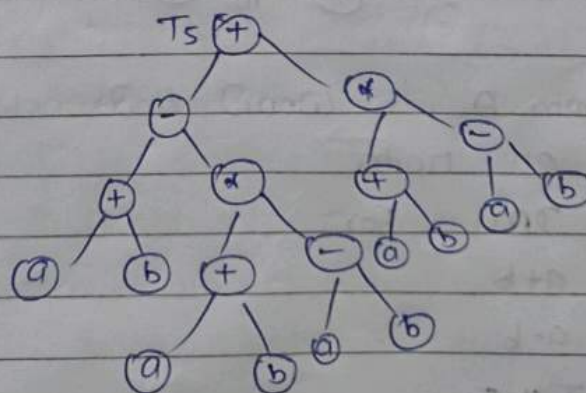
• Step 3: $T_3 = T_1 * T_2$



• Step 4: $T_4 = T_1 - T_3$

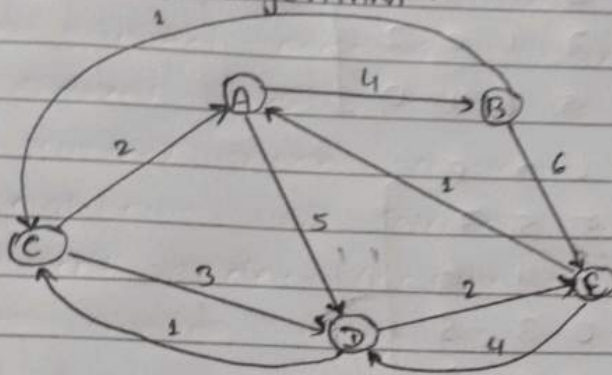


• Step 5: $T_5 = T_4 + T_3$



• Section - C

1. Solve the following graph in accordance with the Warshall's algorithm.



Solⁿ

$$D^0 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & \infty & 5 & \infty \\ \infty & 0 & 1 & \infty & 6 \\ 2 & \infty & 0 & 3 & \infty \\ \infty & \infty & 1 & 0 & 2 \\ 1 & \infty & \infty & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D^1 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & \infty & 5 & \infty \\ \infty & 0 & 1 & \infty & 6 \\ 2 & 6 & 0 & 3 & \infty \\ \infty & \infty & 1 & 0 & 2 \\ 1 & 5 & \infty & 4 & 0 \end{bmatrix} \end{matrix}$$

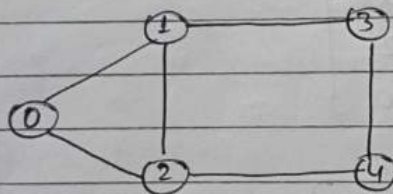
$$D^2 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 10 \\ \infty & 0 & 1 & \infty & 6 \\ 2 & 6 & 0 & 3 & 12 \\ \infty & \infty & 1 & 0 & 2 \\ 1 & 5 & 6 & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D3 = \begin{matrix} & A & B & C & D & E \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 10 \\ 3 & 0 & 1 & 4 & 6 \\ 2 & 6 & 0 & 3 & 12 \\ 3 & 7 & 1 & 0 & 2 \\ 1 & 5 & 6 & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D4 = \begin{matrix} & A & B & C & D & E \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 7 \\ 3 & 0 & 1 & 4 & 6 \\ 2 & 6 & 0 & 3 & 5 \\ 3 & 7 & 1 & 0 & 2 \\ 1 & 5 & 5 & 4 & 0 \end{bmatrix} \end{matrix}$$

$$D5 = \begin{matrix} & A & B & C & D & E \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 4 & 5 & 5 & 7 \\ 3 & 0 & 1 & 4 & 6 \\ 2 & 6 & 0 & 3 & 5 \\ 3 & 7 & 1 & 0 & 2 \\ 1 & 5 & 5 & 4 & 0 \end{bmatrix} \end{matrix}$$

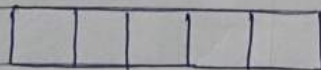
2. Solve the following graph with DFS with BFS.



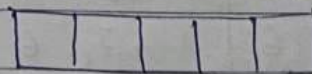
Ans

B.F.S : (Breadth First search)

STEP 1: initially Queue and visited array are empty.

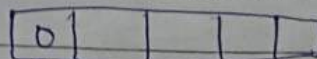
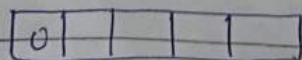


visited



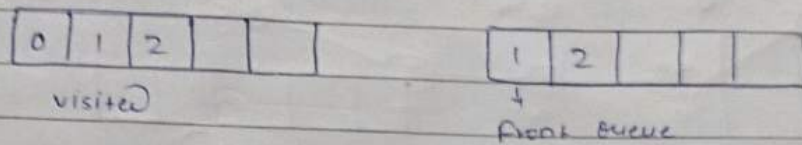
Queue

STEP 2: push 0 to Queue and mark it visited.

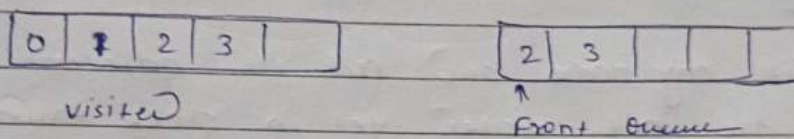


Front Queue

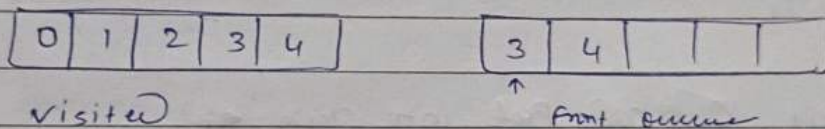
step3: Remove 0 from front of queue and visit the unvisited neighbours and push them into queue.



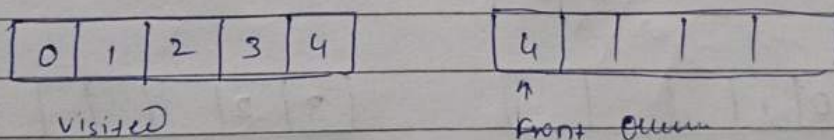
step4: Remove 1 from front and visit unvisited neighbours and push them in queue.



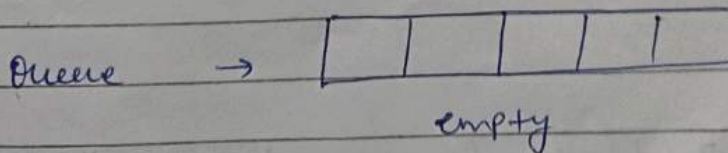
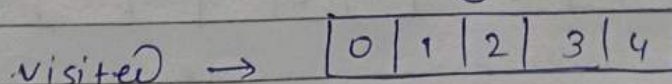
step5: Remove Node 2 and push into queue.



step6: Remove node 3 and push into queue. Neighbours of node 3 is visited.

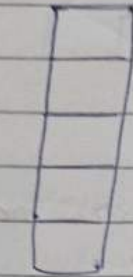


step7: Remove node 4 & push into queue. Node 4 is already in final state.

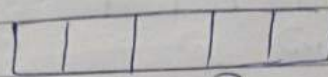


• DFS (Depth First Search):-

STEP 1 initially stack and visited arrays are empty -



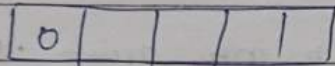
stack



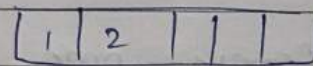
visited

visit any node and put its adjacent nodes which are not visited yet in stack then visit to node which is at top of stack and pop it

STEP 2: visit 0 and put into stack

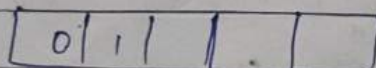


visited

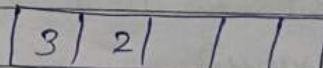


stack

STEP 3:

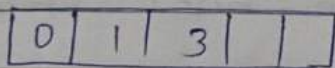


visited

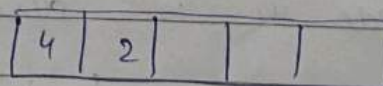


stack

STEP 4:

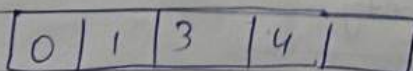


visited

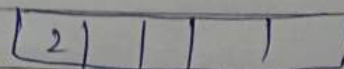


stack

STEP 5:



visited



stack

STEP 6

0 | 1 | 3 | 4 | 2

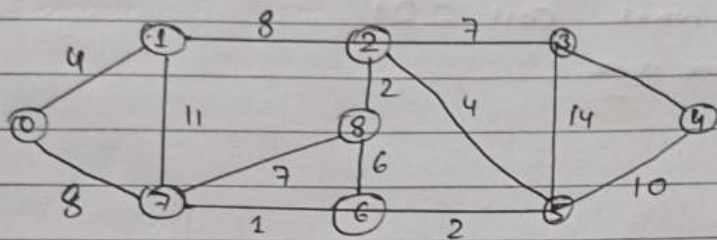
visited

Stack

Stack

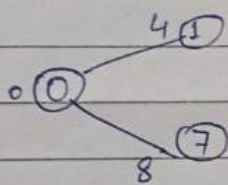
3. Find out minimum cost spanning tree using dijkstra algorithm.

Ans

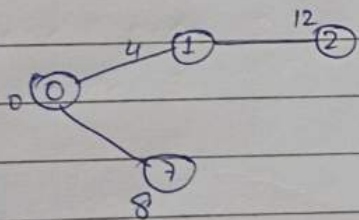


Ans

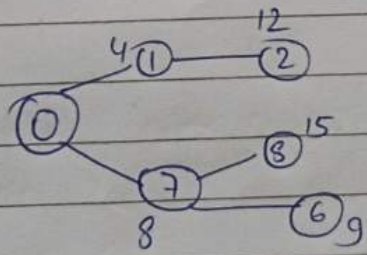
STEP 1



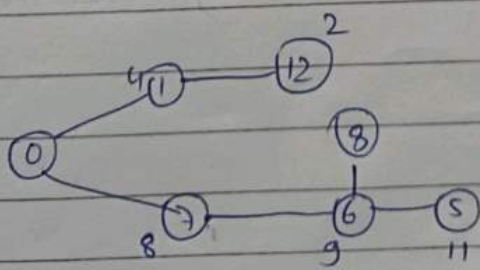
STEP 2

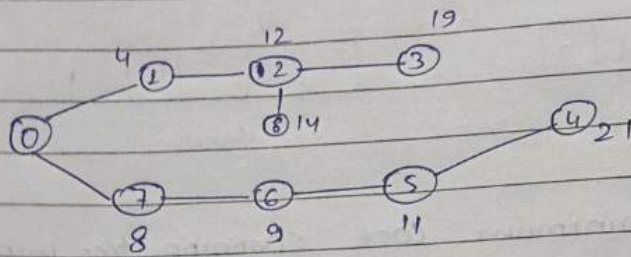


STEP 3



STEP 4



Steps

Total Minimal Cost = 21

from source nodeThe end~~Suby~~
~~2/01/2024~~~~Calced~~